

E-COMMERCE • DATA • RAG • OPERATIONS

电商 AI 商品文案生成与智能导购助手

真实商品数据驱动的双角色电商业务闭环

数据采集

前后端闭环

RAG 导购

离线可演示

项目组（成员姓名待补充） • 2026-07-15

30 分钟讲解路线

01 • 项目与架构 | 6 min

问题背景、目标边界、双角色功能与总体架构。

02 • 数据采集 | 6 min

苏宁主链路、京东实验复盘、数据质量与阶段结果。

03 • 前后端闭环 | 7 min

前端三态、88 个路由、订单与售后、14 张业务表。

04 • RAG 与 AI | 7 min

上下文补全、多路召回、Rerank、问答路由与 Provider。

05 • 演示与验证 | 3 min

端到端演示路径、当前验证结果、风险与改进。

06 • 总结与问答 | 1 min

回答项目价值、技术亮点与下一步。

事实口径：代码是主证据，文档是解释层

当前仓库代码

以 main 分支当前工作区为准；
核对路由、模型、页面、服务、
配置与部署文件。

三份技术文档

用于章节组织、历史试验数据和
原理解释；不直接替代当前实现。

本轮实际验证

前端生产构建通过；后端测试因
当前 .venv 缺少 Flask / pytest 未
能执行。

答辩表述规则

“当前已实现”只用于代码可证实能力；“阶段结果”用于文档记录数据；“下一步”用于尚未完成或尚未验证的内容。

为什么做：电商业务中有两类高成本

商家侧：内容与运营链路分散

- 商品标题、卖点、详情与直播脚本反复整理
- 评论数据难转化成改进建议
- 商品、订单、客服、营收和知识库分散管理
- 外部模型与采集站点不稳定，演示风险高

用户侧：购物决策信息碎片化

- 规格、价格、评价和售后需要跨页面比较
- 自然语言问题难直接映射到商品数据
- 下单、支付、物流、评价和售后需要闭环
- 单轮问答无法理解“这款呢”“还有别的吗”

项目定位：不是单点 AI Demo，而是“数据 → 决策 → 交易 → 运营”的完整系统

项目目标与边界

双角色

用户前台 + 商家后台

6 类

AIProvider 统一能力接口

14 张

当前 ORM 业务数据表

88 条

当前 Flask 路由装饰器

已实现边界：商品浏览与管理、购物车、订单/售后、用户与客服、营收看板、知识库、问答记录、采集任务、文案/导购/评论/直播脚本。

工程边界：课程项目强调可复现与可演示；默认 Mock AI，PostgreSQL 不可用时可降级 SQLite，Redis 属于可选加速层。

不夸大：没有把当前仓库描述成生产级平台；模型评测、监控审计、真实支付与大规模压测仍是后续工作。

当前功能地图：一套系统连接两类角色

用户端

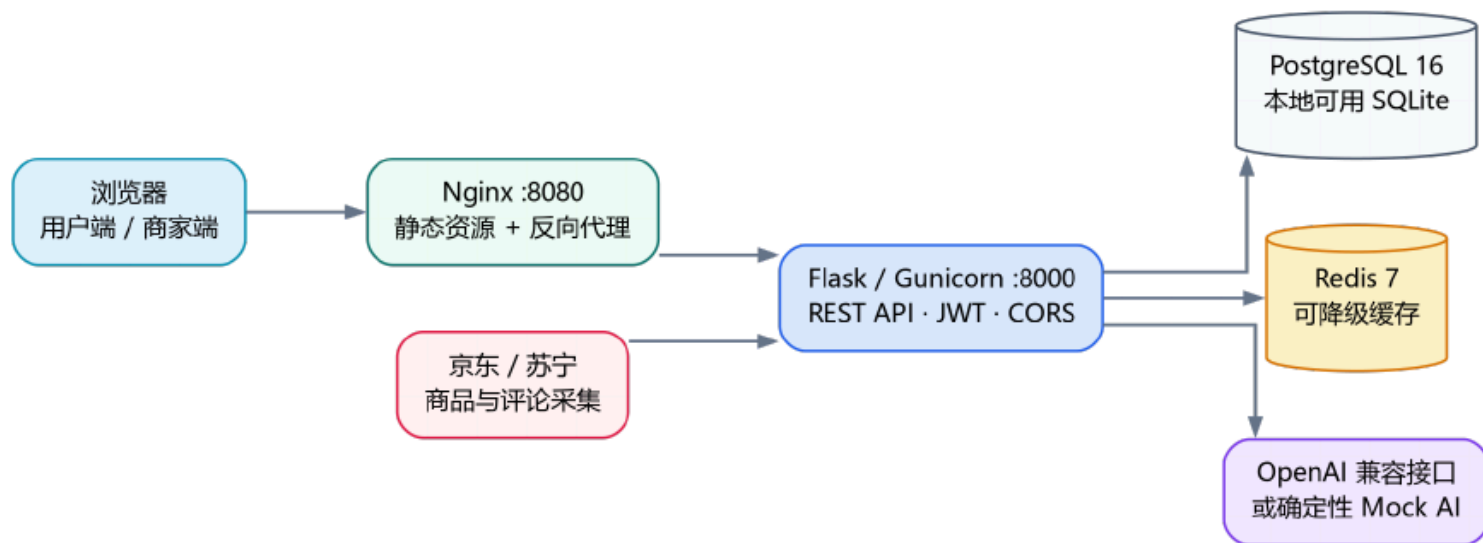
商品浏览 / 搜索 / 收藏 / 历史 购物车 / 下单 / 支付 / 收货 地址 / 评价 / 图片视频 / 退换货 RAG 问答 / 流式输出 / 客服会话

商家端

营收看板 / 导出 / TOP 商品 商品 / 订单 / 用户 / 客服管理 文案 / 评论分析 / 直播脚本 知识库 / QA 统计 / 采集任务

24 个 Vue 文件 • 23 个业务组件 • App.vue 统一三态入口

总体架构：从浏览器到数据与模型



Vue 3 · TypeScript · Vite

Flask · SQLAlchemy · JWT

PostgreSQL / SQLite · Redis

来源：当前 App.vue、backend/app.py、database.py、docker-compose.yml

数据采集在系统中的位置

9 类 SuningCrawler 映射覆盖

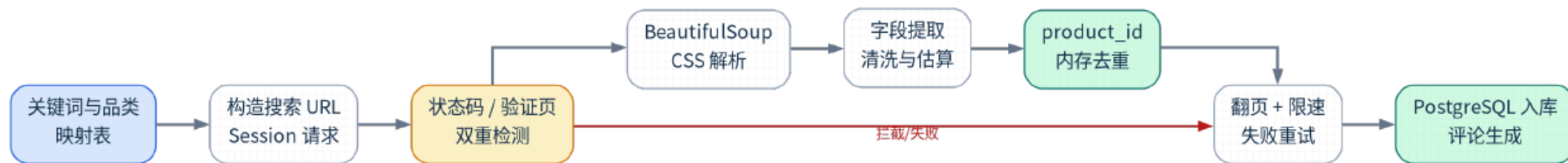
6 类 crawl API 一键预设

异步任务 + 进度

当前代码事实

- /api/crawl/* 的管理链路实际实例化 **SuningCrawler** • 自定义关键词最多 20 个，每关键词最多 5 页 • 任务记录 pending / running / completed / failed 与错误列表 • 入库时按 platform=suning + product_id 去重或更新 • JdCrawler 文件仍存在，但没有接入当前 CrawlManager 主执行链路

当前采集流水线：可观察、可去重、可降级



请求层

Session、UA/Cookie、限速、状态码与验证页检测。

解析层

BeautifulSoup + CSS 选择器，字段回退、清洗与估算。

存储层

商品更新/创建、评论生成、事务提交与任务统计。

来源：当前 suning_crawler.py、crawl_manager.py、crawl_routes.py

苏宁主链路为什么能工作

解析策略

1. 搜索关键词映射到业务分类
2. 商品名采用主选择器 → 图片 alt 的回退
3. 价格按品牌 / 关键词 / 类目区间估算
4. 评价数驱动评分推断与评论分布
5. product_id 控制跨页和数据库去重

工程取舍

- 优先稳定获取商品骨架，而不是追求所有动态字段
- 页面变化会影响选择器，因此必须保留错误与任务状态
- 估算字段不能伪装成平台原始字段，答辩中明确说明
- 当前实现适合课程演示与数据初始化，不等同商业采集服务

关键结论：成功不只来自“请求能返回”，还来自字段回退、质量修复、去重与任务可观察性。

覆盖范围与阶段结果：明确区分当前代码与历史记录

当前代码的 9 类映射

数码电子



6 关键词

2000+

爬虫文档记录的阶段商品量

100%

文档记录的阶段采集成功率

化妆品



5 关键词

不能混淆的口径 · 2000+ 与 100% 来自 crawler-doc 的阶段记录
· 当前仓库未提供可核验的生产数据库快照 · 所以 PPT 不把该

母婴用品



4 关键词

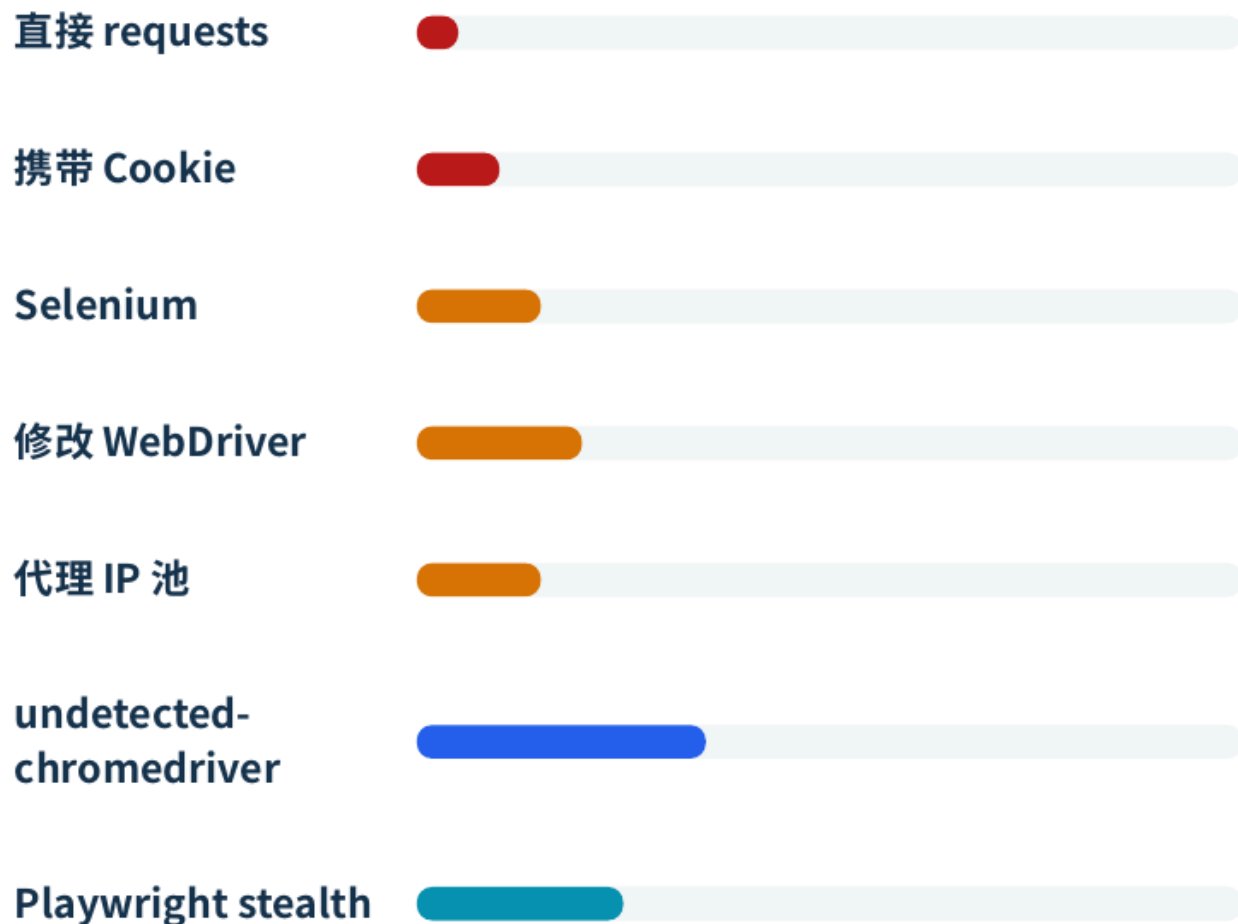
数字称为“本轮实时数据库统计” · 真实演示应以 /api/crawl/
stats 返回值为现场口径

宠物用品



4 关键词

京东实验复盘：失败也形成架构决策



四层阻力

CDN/WAF、验证码、动态渲染/加密、设备指纹与行为分析。

当前决策

JdCrawler 保留为实验；主任务链路采用苏宁。

数据来自 crawler-doc 历史试验。

数据质量：把“不完美来源”变成“可用输入”

采集层

名称长度校验、URL 规范化、验证页检测、跨页 seen_ids 去重。

入库层

platform + product_id 查询；存在则更新，不存在则创建并生成评论。

消费层

评分、评论、类目和知识条目进入 RAG；异常时走 AI/Mock 兜底。

质量风险仍然存在：价格和销量中的部分字段包含估算/生成逻辑；答辩要说明它们用于课程数据完整性和功能演示，不应宣称为平台实时交易数据。

改进方向：为字段增加 provenance（原始/推断/生成）、采集时间、选择器版本与质量分，建立可追踪数据血缘。

前端架构：App.vue 的三态入口

login 未登录统一入口

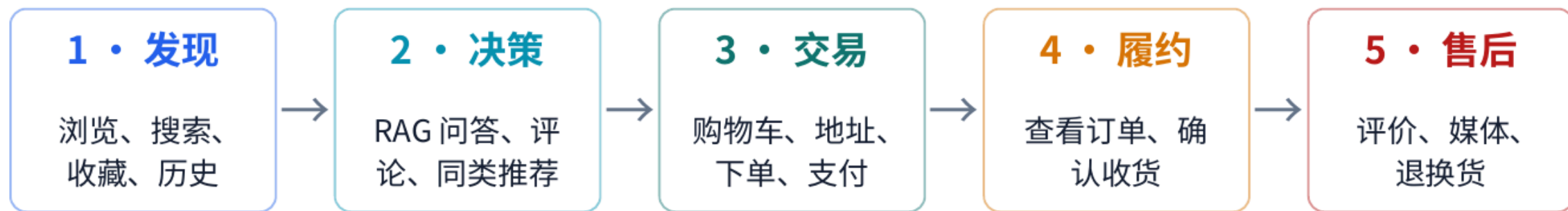
user 用户前台布局

merchant 商家后台布局

组件组织

- App.vue 根据登录态与角色渲染 UserLayout / MerchantLayout
- 业务页面使用异步组件加载，降低首屏一次性加载成本
- api.ts 统一封装请求与 Bearer Token
- 用户端围绕商品与交易，商家端围绕运营与 AI 工具
- 当前生产构建通过，但主 chunk 约 680 KB，仍有拆分空间

用户端闭环：从发现商品到售后



用户不是“问完就走”：问答结果可以继续进入商品、订单、评价与客服链路，数据也会反哺商家端统计。

商家端闭环：运营、内容与服务协同

经营监控

营收 KPI、日趋势、订单趋势、分类分布、TOP 商品与导出。

交易运营

商品 CRUD/媒体、订单搜索发货、退换货处理、用户管理。

AI 内容

文案生成、评论分析、直播脚本、导购与交叉推荐。

知识运营

知识条目 CRUD、按商品自动构建、分类管理与相似度检索。

用户洞察

QA 类型统计、问答记录、来源跟踪与用户咨询热点。

服务协同

客服线程、未读数、商家回复、商品上下文与消息已读。

后端接口层：88 条路由由 7 个域承载

31

商家域路由

28

用户域路由

11

公共 AI/商品域

9

认证与上传域

7

客服域

7

采集域

2

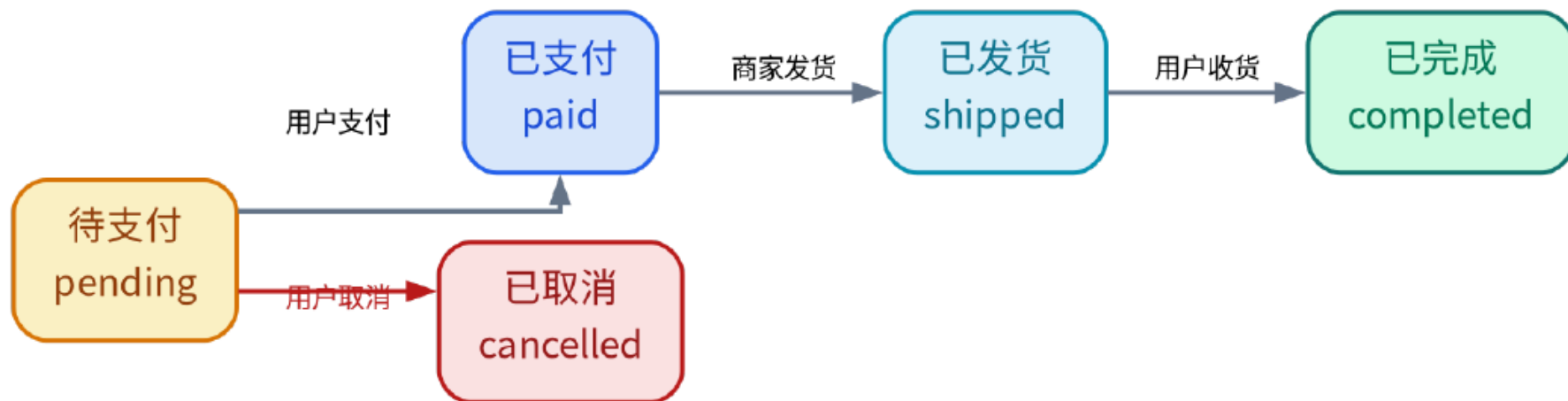
OpenAPI 文档

1

健康检查

JWT 由 Flask-JWT-Extended 签名；服务端 `require_auth` / `require_merchant` 才是权限边界，前端菜单隐藏只负责交互体验。

订单状态机：交易闭环已扩展到退换货



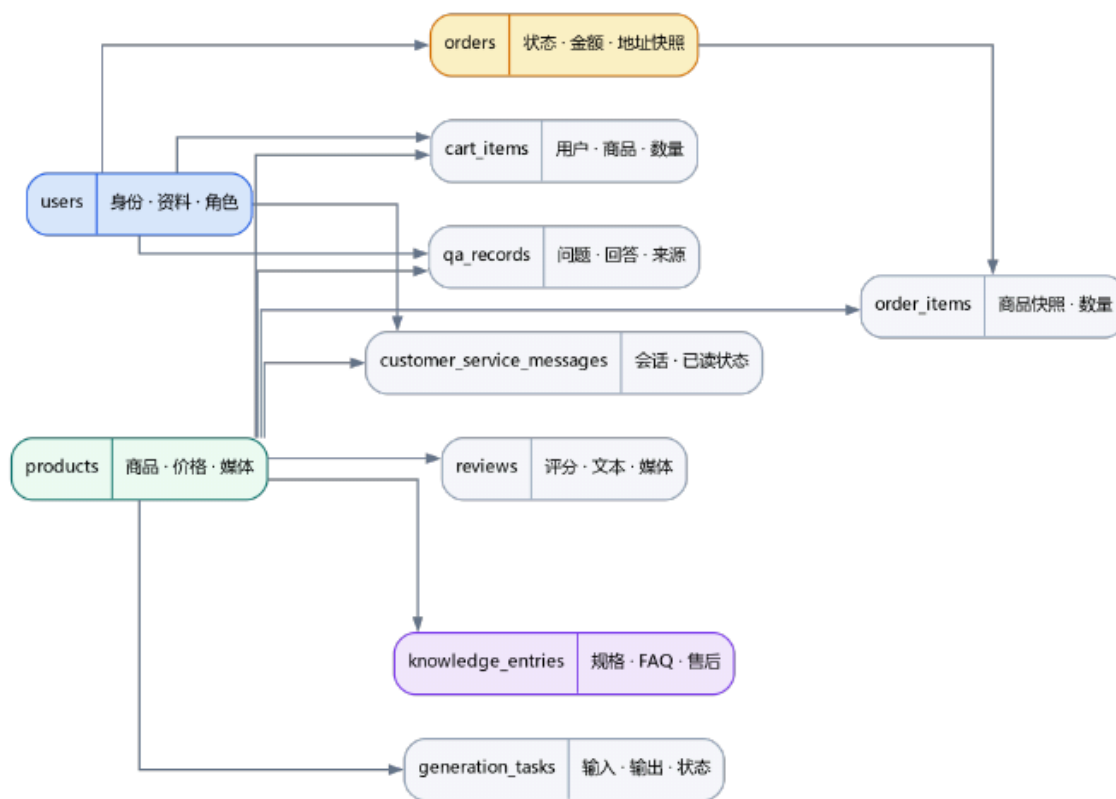
主链路

pending → paid → shipped → completed; 每一步由不同角色触发并记录时间。

售后链路

paid/shipped/completed 可申请 returning; 填写退货单号后由商家处理为 returned。

数据模型：14 张表支撑六个业务域



账号域 User / Address / Favorite / History

交易域 Cart / Order / OrderItem

商品域 Product / Review

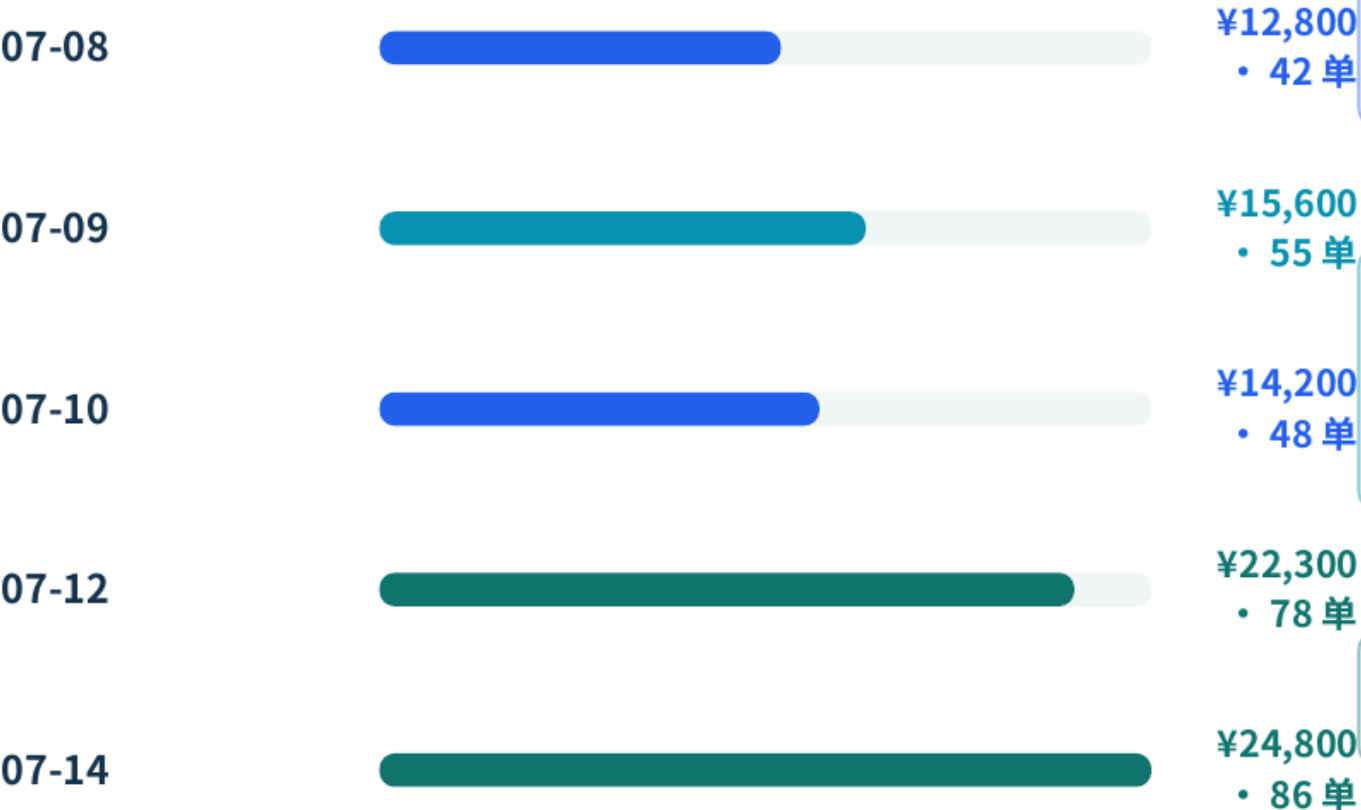
AI 域 GenerationTask / RecommendationLog

知识域 KnowledgeEntry / QARecord

客服域 CustomerServiceMessage

营收看板：从订单事实生成经营视图

文档示例：7 日营收 / 订单趋势



KPI

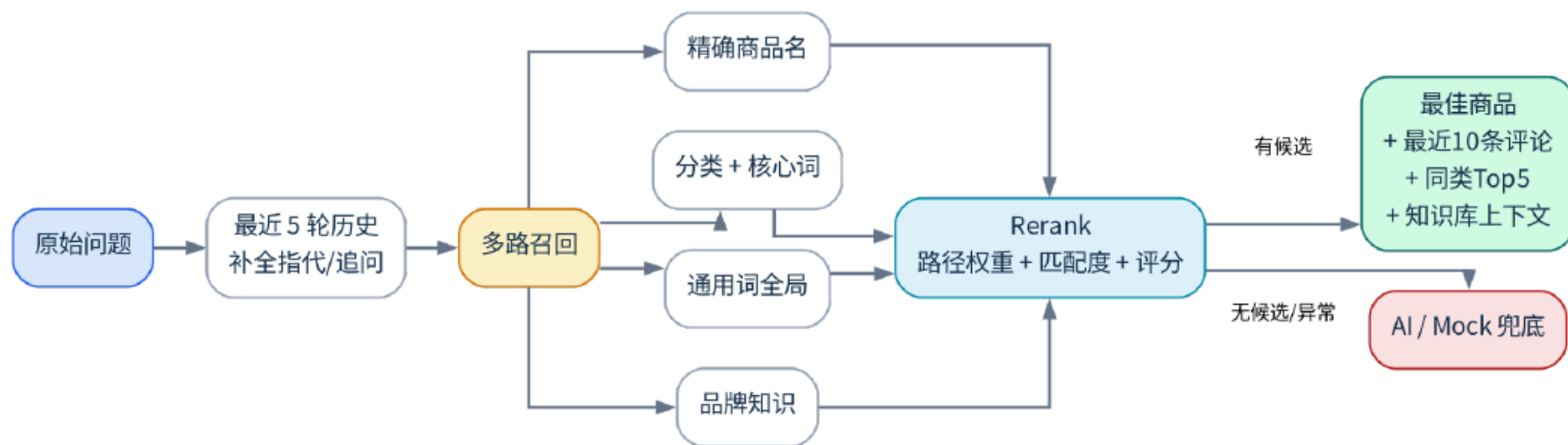
营收 / 订单 / 客单价

趋势

日营收 + 日订单

结构

当前 RAG：不再只是“四级搜索”



上下文 · 最近 5 轮补全

检索 · 多路召回 + Rerank

回答 · 数据聚合 + 兜底

长对话与多路召回：解决“这款呢”

历史补全

用户：推荐 300 元内的蓝牙耳机 系统：返回商品 A 与备选
用户：“**这款续航怎么样？**” 系统：从上轮回答提取商品名 / product_id，再构造完整问题

候选合并

Path 1：精确商品名 Path 2：分类 + 核心关键词 Path 3：通用关键词全局召回 Path 4：品牌知识 / 知识条目 → 去重后按路径权重、匹配度、评分重新排序

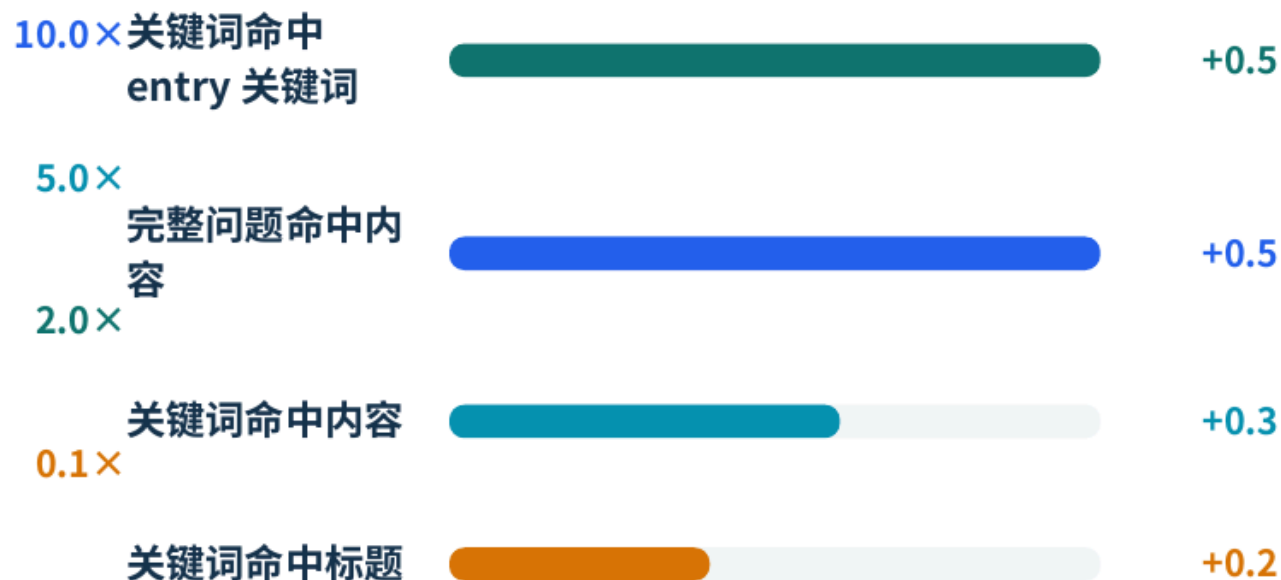
这部分是当前代码相对 rag-ai-doc 的重要演进：文档仍保留四级串行解释，代码已经加入长对话和多路召回。

检索评分：可解释的规则先保证稳定

商品召回权重（文档口径）



知识条目相似度加分



优势是可解释、可离线、容易调试；局限是语义泛化能力弱，未来可引入向量召回与离线评测，但必须保留规则检索作保底。

问题类型路由：9 类回答构建器

推荐 recommend

需求、预算、评分与同类商品。

价格 price

当前价、原价、同类价格带。

品牌 brand

商品真实品牌 + 内置品牌知识。

评价 review

评分、最近评论与口碑摘要。

比较 compare

主商品与同类候选对比。

尺寸 size

规格字段与知识条目。

功能 function

卖点、规格与评论反馈。

售后 after_sale

退换、保修与 FAQ 上下文。

通用 general

数据驱动摘要或 AI/Mock 兜底。

回答如何生成：数据优先，模型兜底



命中商品

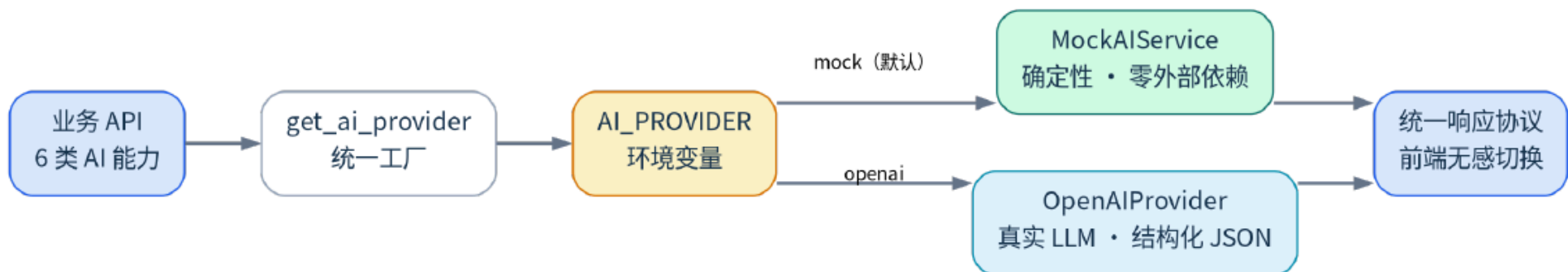
按问题类型拼装数据驱动回答；返回主商品卡片和最多 3 个相关商品。

未命中 / 检索异常

调用 `AIProvider.guide_qa`；再失败时返回可操作的自然语言提示。

每次问答写入 `QARecord`，source 区分 `data_rag` 与 `ai_fallback`，便于后续统计和审计。

Provider 设计：默认稳定演示，按配置接真实模型



MockAIService

默认代码路径；零密钥、确定性、可覆盖文案/推荐/问答/评论/直播脚本。

OpenAIProvider

OpenAI-compatible Chat Completion；模型、Key、Base URL 由环境变量提供。

配置注意：当前 get_ai_provider 只识别 AI_PROVIDER=openai；README/.env 的 deepseek 值与代码存在漂移，需统一。

知识库与 QA 形成运营反馈环

知识库管理

- spec / faq / after_sale 等分类条目
- 支持增删改查、启用状态和商品关联
- 从商品规格、卖点、售后规则自动构建
- 规则相似度检索并返回 score

问答统计

- 记录 user / product / question / answer
- 标记 question_type 与 answer source
- 商家查看类型分布与最近问答
- 识别高频问题并反向补齐知识库

问答不是终点：用户问题 → QA 统计 → 知识补齐 → 下一次回答更完整

端到端闭环：一条答辩演示串起全系统

1 · 用户提问

“300 元内蓝牙耳机怎么选？”

2 · RAG 检索

上下文、多路召回、评论与知识聚合。

3 · 用户下单

商品卡片 → 购物车 → 地址 → 支付。

4 · 商家履约

订单看板 → 发货 → 用户确认收货。

5 · 用户评价

星级、文字、图片/视频与售后。

6 · 评论分析

情感、关键词、差评痛点与建议。

7 · 内容生产

文案生成与直播脚本用于再营销。

8 · 经营复盘

营收、订单、TOP 商品、QA 与客服。

演示建议：控制在 4-5 分钟，预先准备 Mock 数据和两个账号

工程验证、已知问题与下一步

本轮确认

✓ 前端 npm run build 成功 ✓ 当前代码静态统计：88 路由、14 模型、24 Vue 文件 ✓ 12 个测试函数存在于 test_api / test_auth ✓ Compose 编排 PostgreSQL、Redis、后端、前端

未完成验证 / 风险

- 当前 .venv 缺少 Flask 与 pytest，后端测试未执行
- AI_PROVIDER 文档配置与工厂代码存在漂移
- 主前端 chunk 约 680 KB，需进一步拆分
- 采集字段包含估算值，需补充数据血缘标记

近期

修复 Provider 配置一致性；恢复后端依赖并跑通 12 个测试。

中期

加入向量召回、离线问答集、命中率/无答案率与质量评测。

长期

完善监控、审计、内容安全、支付沙箱和采集 provenance。

总结：项目价值在于把 AI 放进真实业务流程

数据有来源

采集、清洗、入库、评论和知识条目形成可消费的数据基础。

系统有闭环

用户决策与交易、商家履约与运营、客服与售后彼此联通。

AI 有边界

数据优先、Mock 保底、真实模型可切换；结果可记录、可解释、可改进。

从“能生成”走向“可决策、可交易、可运营”

Q & A